

TEMA 3

1. Introducción: ¿Qué es el Proceso de Software?

Empieza definiendo el concepto base para demostrar dominio.

- **Definición:** Es un conjunto de actividades que conducen a la creación de un producto de software .
 - **Actividades Fundamentales (comunes a todos los procesos):**
 1. **Especificación:** Qué debe hacer el software y sus restricciones .
 2. **Diseño e implementación:** Producir el software según la especificación .
 3. **Validación:** Asegurar que hace lo que el cliente quiere .
 4. **Evolución:** Adaptarlo a necesidades cambiantes .
 - **Punto clave para el oral:** No existe un proceso "ideal". Depende de las personas, la organización y el tipo de sistema (crítico vs. de negocios) .
-

2. Modelos Tradicionales y Prescriptivos

Aquí está el núcleo del examen. Debes saber explicar cómo funciona cada uno y sus pros/contras.

A. Modelo en Cascada (Lineal Secuencial)

- **Cómo funciona:** Fases separadas y secuenciales. El resultado de una fase es un documento que "firma" el inicio de la siguiente.
- **Fases:** Requerimientos - Diseño Implementación/Pruebas unitarias - Integración/Pruebas de sistema - Mantenimiento.
- **Cuándo usarlo:** Solo cuando los requerimientos se entienden bien y no cambiarán (ej. grandes proyectos de ingeniería).
- **Desventaja crítica:** Es inflexible. "Congela" los requerimientos prematuramente; si el cliente cambia de opinión al final, es muy costoso arreglarlo.

B. Modelo de Prototipos

- **Concepto:** Se usa para dar al usuario una vista preliminar cuando no conoce bien los requisitos detallados . Es "prueba y error" .
- **Tipos de Prototipos:**
 - **Desechable:** Se hace para aclarar dudas y definir la interfaz, luego se tira .
 - **Evolutivo:** Se construye un núcleo y se refina hasta convertirse en el sistema final (ojo: puede quedar mal estructurado o con "parches") .
- **Ventaja:** Reduce el riesgo de construir algo que el usuario no quiere .

C. Desarrollo Rápido de Aplicaciones (RAD)

- **Concepto:** Modelo lineal secuencial pero de "alta velocidad" (ciclo de 60 a 90 días) .
- **Clave:** Se basa mucho en la **reutilización de componentes** y herramientas de 4ta generación .
- **Restricción:** Requiere modularización (equipos trabajando en paralelo) y compromiso fuerte de cliente/desarrollador. Si no se puede modularizar, falla .

D. Ingeniería de Software Basada en Componentes (CBSE)

- **Enfoque:** En lugar de programar desde cero, se integra software existente (COTS o componentes reutilizables) .
 - **Diferencia en el proceso:** Tiene etapas únicas como "Análisis de componentes" (buscar qué sirve) y "Diseño con reutilización" .
-

3. Modelos Iterativos y Evolutivos

Fundamental para contrastar con la rigidez de la cascada.

A. Entrega Incremental

- **Estrategia:** Se identifican los servicios y se priorizan. Se entrega primero el incremento con la funcionalidad más crítica .
- **Ventaja:** El cliente recibe valor rápido y puede probar el sistema temprano (feedback real) .

B. Modelo en Espiral (Gestión de Riesgos)

- **Concepto clave:** Representa el proceso como una espiral donde cada ciclo considera explícitamente el **RIESGO** .
- **Sectores del ciclo:**
 1. Definición de objetivos.
 2. Evaluación y reducción de riesgos (ej. hacer un prototipo si hay dudas).
 3. Desarrollo y validación.
 4. Planificación de la siguiente fase .
- **Variante WINWIN:** Agrega actividades de "negociación" al inicio para equilibrar las condiciones de victoria de todos los involucrados .

C. Proceso Unificado de Rational (RUP)

- **Qué es:** Un proceso moderno diseñado junto con UML, iterativo e incremental .
- **Perspectiva Dinámica (Fases):**
 1. **Inicio:** Establecer el caso de negocio (¿es viable?) .
 2. **Elaboración:** Comprender el dominio, arquitectura base y riesgos .

3. **Construcción:** Diseño, programación y pruebas del sistema operativo .
 4. **Transición:** Mover el sistema al usuario (despliegue) .
- **Perspectiva Estática (Flujos de trabajo):** Actividades como modelado de negocio, requisitos, análisis, etc., que ocurren durante todas las fases con distinta intensidad .
-

4. Metodologías Ágiles (Foco en XP)

Si te preguntan por ágiles, céntrate en XP que es lo más detallado en el texto.

Programación Extrema (XP)

- **Objetivo:** Aumentar productividad y adaptarse a requisitos cambiantes mediante "pequeñas entregas" .
 - **Valores:** Comunicación, Sencillez (hacer solo lo necesario hoy), Retroalimentación (pruebas constantes), Valentía .
 - **Prácticas clave (Menciona al menos 4-5):**
 - **Juego de planificación:** Diálogo constante negocio-técnico .
 - **Diseño sencillo y Refactorización:** Mejorar el código sin cambiar su funcionalidad externa .
 - **Programación por parejas:** Dos personas, un ordenador. Aumenta calidad .
 - **Pruebas (TDD):** Se escriben las pruebas antes o durante el código .
 - **Cliente en casa:** Un cliente real trabaja con el equipo .
 - **Integración continua:** Integrar y probar el código al menos una vez al día .
 - **Semana de 40 horas:** Evitar el agotamiento .
 - **Otras ágiles mencionadas:**
 - **Kanban:** Sistema de flujo visual ("Justo a tiempo"), originado en Toyota. Se enfoca en producir lo necesario en el momento necesario y limitar el trabajo en curso .
 - **ASD (Desarrollo Adaptable):** Enfocado en sistemas complejos, tolerante a cambios y aprendizaje continuo .
-

5. Métodos Formales y Herramientas

Métodos Formales

- **Definición:** Uso de representaciones matemáticas rigurosas (lógica, teoría de conjuntos) para especificar y verificar software .
- **Uso:** Sistemas críticos (seguridad, vidas humanas, aeroespacial) donde el costo del fallo es altísimo .

- **Pros:** Calidad asegurada matemáticamente, sin ambigüedad .
- **Contras:** Muy caro, lento, requiere expertos matemáticos, difícil comunicación con el cliente .

Herramientas CASE (Computer-Aided Software Engineering)

- **Definición:** Software para ayudar en las actividades del proceso (diseño, programación, pruebas) .
 - **Clasificación (importante para diferenciar):**
 1. **Herramientas:** Apoyan tareas individuales (ej. un compilador, editor) .
 2. **Bancos de trabajo:** Conjunto de herramientas integradas para una fase (ej. banco de trabajo de diseño) .
 3. **Entornos:** Apoyan el proceso completo (ej. Entornos centrados en procesos) .
-

6. Tendencias Tecnológicas (Contexto Moderno)

Úsalo como cierre o si te preguntan sobre actualidad.

- Menciona que la ingeniería de software ahora se apoya en **Sistemas Distribuidos, Big Data** (Hadoop, NoSQL como Cassandra), **Arquitectura Orientada a Servicios (SOA)** y **Computación Ubicua/Móvil** .
 - Destaca la importancia de la **Arquitectura de Software** (estructurar componentes para calidad) y el uso de **Patrones de Diseño** (soluciones reutilizables a problemas comunes) .
-

Consejos para tu Defensa Oral (El "plus" de Gemini):

1. **Comparaciones:** Es probable que te pregunten "*¿Diferencia entre Espiral y Cascada?*".
 - *Respuesta:* Cascada es lineal y asume requisitos fijos; Espiral es iterativo y su motor principal es la gestión de riesgos.
2. **Si te preguntan por RUP:** No olvides mencionar que separa las **Fases** (tiempo/negocio) de los **Flujos de Trabajo** (actividades técnicas). Es su innovación principal .
3. **Sobre XP:** Si te preguntan por qué funciona, di que maneja la "curva de costo del cambio". En métodos tradicionales, corregir un error tarde es carísimo; en XP, gracias a las pruebas y refactorización, el costo se mantiene plano .
4. **Diagramas Mentales:** Visualiza el gráfico del Espiral (bucles), la Cascada (escalones hacia abajo) y el RUP (fases con ondas de actividad) mientras hablas.

TEMA 4

1. Concepto de Educción (Elicitation)

Para empezar, debes definir qué es este proceso.

- **Definición:** Educar significa "sacar algo de algo". Es el proceso de descubrir, revisar, articular y entender las necesidades de los usuarios y las restricciones del sistema.
- **Pasos del procedimiento:**
 1. Identificar fuentes de información.
 2. Preguntar para comprender necesidades.
 3. Analizar buscando inconsistencias.
 4. Confirmar con el usuario.

2. Problemas en la Educción

Es vital mencionar por qué es difícil este proceso. Se agrupan en tres categorías:

- **Problemas de Alcance:** Definir mal los límites del sistema (demasiada o muy poca información).
- **Problemas de Comprensión:** Diferencias de lenguaje entre usuarios y técnicos, ambigüedad en la expresión de necesidades.
- **Problemas de Volatilidad:** Los requisitos cambian con el tiempo porque el negocio evoluciona o el usuario aprende durante el proceso.

3. Técnicas de Educción de Requisitos

Este es el grueso del tema. Debes poder explicar brevemente las más importantes.

A. Técnicas Tradicionales y de Apoyo

- **Muestreo:** No es una técnica de educación per se, sino de selección sistemática de elementos (personas o documentos) para aplicar otras técnicas.
- **Entrevista:** Conversación dirigida con propósito específico.
 - *Planificación:* Leer antecedentes, establecer objetivos, seleccionar entrevistados.
 - *Tipos de preguntas:* **Abiertas** (ricas en detalle, difíciles de analizar) y **Cerradas** (respuestas limitadas, fáciles de tabular).
 - *Estructuras:* Pirámide (de específico a general), Embudo (de general a específico) y Diamante (combinación).
- **Cuestionarios:** Útil para grupos numerosos o dispersos geográficamente.
 - Usa escalas como la **Escala de Likert** para medir actitudes (ej: "Muy de acuerdo" a "Muy en desacuerdo").
 - *Ventaja:* Rápido y tabulable. *Desventaja:* Poca profundidad.

B. Técnicas Grupales y Creativas

- **Brainstorming (Lluvia de ideas):** Técnica grupal donde se generan ideas libremente sin críticas inmediatas. Ayuda a abrir la mente y evitar centrarse en detalles prematuros.
- **JAD (Joint Application Design):** Diseño conjunto. Reúne a usuarios, clientes y analistas en sesiones intensivas.
 - *Objetivo:* Crear una visión compartida y acelerar decisiones.
- **Arreglos-Q (Q-Sort):** El usuario ordena tarjetas con conceptos en pilas según una distribución normal (ej: importante vs. no importante).

C. Técnicas Orientadas al Sistema y Contexto

- **Prototipos:** Construcción rápida de un sistema (funcional o no) para que el usuario refine sus requisitos mediante la experimentación.
- **PIECES:** Acrónimo para explorar requisitos en 6 categorías: Rendimiento, Información, Economía, Control, Eficiencia y Servicios.
- **Etnografía / STROBE:** Observación directa del entorno del usuario (lenguaje corporal, oficina, flujo de trabajo real vs. teórico).

D. Casos de Uso (Fundamental)

- **Definición:** Técnica basada en escenarios que describe la interacción entre un **Actor** y el **Sistema**.
- **Elementos:**
 - **Actor:** Quien inicia o recibe la acción.
 - **Escenario:** Secuencia de pasos.
- **Relaciones:**
 - **Inclusión (<<include>>):** Reutilizar pasos comunes en varios casos.
 - **Extensión (<<extend>>):** Agregar pasos opcionales a un caso base.
 - **Generalización:** Herencia entre casos de uso o actores.

4. Definición y Tipos de Requerimientos

- **Definición de Requisito:** Una condición o capacidad que el sistema debe cumplir para resolver un problema del usuario.
- **Niveles de Abstracción:**
 - **Requerimientos de Usuario:** Lenguaje natural, comprensibles para el cliente.
 - **Requerimientos del Sistema:** Especificación detallada y técnica para desarrolladores.
- **Clasificación Funcional:**
 1. **Funcionales:** Servicios que el sistema debe proveer (lo que hace).

2. **No Funcionales:** Restricciones sobre los servicios (seguridad, rendimiento, portabilidad).
3. **Del Dominio:** Derivados del entorno de aplicación (ej: normas legales).

5. Calidad de los Requisitos (Características)

Para que un requisito sea válido, debe ser:

- **Completo:** Contiene toda la información necesaria.
- **Correcto:** Describe precisamente la funcionalidad deseada.
- **Realizable:** Técnicamente posible.
- **Necesario:** El cliente realmente lo necesita.
- **Priorizable:** Distinguir lo esencial de lo deseable.
- **No Ambiguo:** Solo una interpretación posible.
- **Verificable:** Se puede probar (testear).
- **Trazable:** Se conoce su origen.

6. Documento de Especificación (IEEE 830)

Es la declaración oficial de qué deben implementar los desarrolladores.

- **Estructura básica:**
 1. **Introducción:** Propósito y alcance.
 2. **Descripción General:** Perspectiva del producto, usuarios, restricciones.
 3. **Requisitos Específicos:** La sección más importante. Detalla funciones, interfaces y rendimiento.

7. Gestión y Trazabilidad

- **Gestión de Cambios:** Proceso formal para evaluar el impacto y costo de los cambios antes de aceptarlos.
- **Trazabilidad:** Capacidad de seguir la vida de un requisito en ambas direcciones (hacia atrás a su origen, hacia adelante a su diseño/código).
 - **Matriz de Trazabilidad:** Herramienta para asegurar que cada requisito tiene casos de prueba asociados (cobertura).

TEMA 5

1. Introducción: ¿Por qué modelamos?

- **Concepto:** Un modelo es una **abstracción** de la realidad. Se omiten detalles para resaltar lo relevante.
 - **Propósito:**
 1. Ayuda a comprender el sistema (es más fácil ver un gráfico que leer 100 páginas de texto).
 2. Sirve de **punto** entre el análisis (qué hacer) y el diseño (cómo hacerlo).
 - **Tipos de Modelos:** Se clasifican según su perspectiva: externa (contexto), de comportamiento (flujo/estados), estructural (datos/clases).
-

2. Modelos de Contexto (El "Afuera" y el "Adentro")

- **Objetivo:** Definir los **límites** del sistema. Distinguir qué es el sistema y qué es el entorno.
 - **Diagrama Arquitectónico:** Muestra el sistema en el centro y las entidades externas (otros sistemas, bases de datos compartidas) con las que interactúa.
 - *Importante:* Define dependencias pero no muestra el detalle de *cómo* se procesan los datos internamente.
-

3. Modelos de Comportamiento (Metodologías Estructuradas)

Aquí se analizan los métodos clásicos (Yourdon) antes de pasar a UML.

A. Diagramas de Flujo de Datos (DFD)

- **Enfoque:** Dirigido por datos. Muestra cómo la información se transforma al pasar por el sistema.
- **Componentes Clave:**
 - **Procesos (Burbujas):** Transforman entradas en salidas (verbo + objeto).
 - **Flujos (Flechas):** Datos en movimiento.
 - **Almacenes (Líneas paralelas):** Datos en reposo.
 - **Terminadores (Rectángulos):** Entes externos (personas, organizaciones).
- **Reglas de consistencia:** Evitar "agujeros negros" (entrada sin salida) y "generación espontánea" (salida sin entrada).

B. Diagramas de Transición de Estados (DTE)

- **Enfoque:** Dirigido por eventos. Ideal para **sistemas de tiempo real** (ej. un microondas, un cajero).
 - **Concepto:** El sistema está en un **Estado** y un **Evento** dispara una transición a otro estado.
-

4. Modelos de Datos (Estructura)

A. Modelo Entidad-Relación (DER)

- Muestra la estructura lógica de los datos. Se usa para bases de datos.
- Elementos: Entidades (Rectángulos), Relaciones (Rombos) y Atributos.

B. Diccionario de Datos

- Es una lista organizada de todos los nombres usados en los modelos (entidades, flujos, procesos) para garantizar que todos en el equipo hablen el mismo idioma y no haya duplicados.
-

5. Modelado Orientado a Objetos (UML)

Este es el estándar moderno. Debes saber diferenciar diagramas estáticos de dinámicos.

A. Diagramas Estáticos (Estructura)

1. **Diagrama de Clases:** El más importante. Describe tipos de objetos y sus relaciones estáticas.
 - **Clase:** Rectángulo con Nombre, Atributos y Operaciones/Métodos.
 - **Relaciones:**
 - *Asociación:* Relación básica (línea).
 - *Generalización (Herencia):* "Es un tipo de..." (Triángulo vacío).
 - *Agregación:* "Es parte de..." (Rombo vacío). El todo es más importante que la parte, pero pueden vivir separados.
 - *Composición:* Agregación fuerte (Rombo lleno). Si muere el todo, muere la parte.
 -
2. **Diagrama de Objetos:** Una "foto" del sistema en un momento dado. Muestra instancias reales (ej: "MiAuto:Automovil").
2. **Diagrama de Componentes:** Muestra la organización física o lógica del código (archivos, librerías .dll, ejecutables) y sus dependencias.
3. **Diagrama de Despliegue (Distribución):** Muestra el **Hardware (Nodos)** y qué software corre en cada máquina (ej: Servidor vs. Cliente).

B. Diagramas Dinámicos (Comportamiento/Interacción)

1. **Diagrama de Secuencia:** Muestra la interacción entre objetos a lo largo del **tiempo**.
 - Clave: **Líneas de vida** (verticales) y **Mensajes** (flechas horizontales).
 -
 2. **Diagrama de Colaboración (Comunicación):** Muestra lo mismo que el de secuencia, pero enfocado en **quién habla con quién** (estructura espacial) y no en el tiempo exacto.
 2. **Diagrama de Actividades:** Flujo de trabajo. Similar a un diagrama de flujo pero soporta **procesos paralelos** (con barras de sincronización).
-

6. Herramientas CASE

- Para finalizar, menciona que el modelado no se hace en papel, sino con software (Herramientas CASE) que permite verificar consistencia, generar código automático y trabajar en equipo.
- Ejemplos: Rational Rose, Enterprise Architect, ArgoUML